Scenario-based problem statements using the `ArrayList` class

1. Library Book Management System

A library wants to maintain a list of book titles currently available for borrowing. The system should allow the librarian to add new books, insert a book at a specified position, update a book title, remove a book that is no longer available, and search for a particular book.

Classes and methods to implement:

- `Library`: Maintain an `ArrayList<String>` of book titles.
- `addBook(String title)`: Add a book.
- `insertBook(int index, String title)`: Insert a book at a specified index.
- `updateBook(String oldTitle, String newTitle)`: Update a book title.
- `removeBook(String title)`: Remove a book.
- `searchBook(String title)`: Check whether a book exists.
- `displayBooks()`: Display all books.
- `getTotalBooks()`: Return the total number of books.
- `Main`: Create an object of `Library` and test all operations.

2. Online Shopping Cart Management System

An e-commerce application maintains a list of product names selected by a customer. The customer should be able to add products, insert a product at a specific index, replace an existing product, remove unwanted products, and search for a product before placing an order.

Classes and methods to implement:

- `ShoppingCart`: Maintain an `ArrayList<String>` of product names.
- `addProduct(String productName)`: Add a product.
- `insertProduct(int index, String productName)`: Insert a product.
- `updateProduct(String oldName, String newName)`: Replace a product.
- `removeProduct(String productName)`: Remove a product.
- `searchProduct(String productName)`: Search for a product.
- `displayProducts()`: Display all products.
- `getTotalProducts()`: Return the number of products.
- `Main`: Test the shopping cart operations.

3. Student Marks Management System

A teacher wants to maintain the marks obtained by students in a test. The system should store marks in an `ArrayList<Integer>`, allow the teacher to add new marks, update marks at a particular index, remove an incorrect entry, and search for a particular mark. It should also calculate the highest, lowest, and average marks.

Classes and methods to implement:

- `MarksManager`: Maintain an `ArrayList<Integer>` of marks.
- `addMarks(int marks)`: Add marks.
- `updateMarks(int index, int marks)`: Update marks at an index.
- `removeMarks(int marks)`: Remove a specified mark.
- `searchMarks(int marks)`: Check whether a mark exists.
- `displayMarks()`: Display all marks.
- `findHighestMarks()`: Return the highest mark.
- `findLowestMarks()`: Return the lowest mark.
- `calculateAverage()`: Calculate the average.
- `Main`: Test all operations.

Assume valid marks range from 0 to 100.

4. Employee Record Management System

A company wants to maintain the names of its employees in a dynamic list. The HR department should be able to add new employees, insert an employee at a specified position, update an employee's name, remove employees who have left the company, and check whether a particular employee is registered.

Classes and methods to implement:

- `EmployeeManager`: Maintain an `ArrayList<String>` of employee names.
- `addEmployee(String name)`: Add an employee.
- `insertEmployee(int index, String name)`: Insert an employee.
- `updateEmployee(int index, String newName)`: Update an employee's name.
- `removeEmployee(String name)`: Remove an employee.
- `searchEmployee(String name)`: Search for an employee.
- `displayEmployees()`: Display all employees.
- `getEmployeeCount()`: Return the total number of employees.
- `Main`: Test all operations.

5. Hospital Appointment Management System

A hospital maintains a list of patient names waiting for appointments. The receptionist should be able to register a new patient, insert an urgent appointment at a specified position, remove a cancelled appointment, update a patient's name, and search for a patient in the appointment list.

Classes and methods to implement:

- `AppointmentManager`: Maintain an `ArrayList<String>` of patient names.
- `addPatient(String name)`: Add a patient at the end.

- `insertUrgentPatient(int index, String name)`: Insert a patient at a specified position.
- `updatePatient(int index, String newName)`: Update a patient entry.
- `cancelAppointment(String name)`: Remove a patient from the list.
- `searchPatient(String name)`: Check whether the patient has an appointment.
- `displayAppointments()`: Display the appointment list.
- `getAppointmentCount()`: Return the number of appointments.
- `Main`: Test all operations.

6. Train Passenger Waiting List

A railway reservation system maintains a waiting list of passenger names. The system should allow passengers to be added to the waiting list, insert a passenger at a specified position when required, remove a passenger who cancels the reservation, search for a passenger, and display the current waiting list.

Classes and methods to implement:

- `WaitingList`: Maintain an `ArrayList<String>` of passenger names.
- `addPassenger(String name)`: Add a passenger.
- `insertPassenger(int index, String name)`: Insert a passenger at a specified position.
- `removePassenger(String name)`: Remove a passenger.
- `searchPassenger(String name)`: Search for a passenger.
- `displayWaitingList()`: Display all passengers.
- `getWaitingCount()`: Return the number of passengers.
- `Main`: Test the waiting-list operations.

7. To-Do List Management Application

A user wants to maintain a list of daily tasks. The application should allow the user to add tasks, insert a task at a specific position, update an existing task, remove completed tasks, search for a task, and display the number of pending tasks.

Classes and methods to implement:

- `TaskManager`: Maintain an `ArrayList<String>` of tasks.
- `addTask(String task)`: Add a task.
- `insertTask(int index, String task)`: Insert a task.
- `updateTask(int index, String newTask)`: Update a task.
- `completeTask(String task)`: Remove a completed task.
- `searchTask(String task)`: Search for a task.
- `displayTasks()`: Display all tasks.
- `getPendingTaskCount()`: Return the number of pending tasks.
- `Main`: Test the application.

8. Movie Watchlist Management System

A user wants to maintain a personal watchlist of movies. The application should allow the user to add movie titles, insert a movie at a specified position, update a movie title, remove a movie after watching it, search for a movie, and display the complete watchlist.

Classes and methods to implement:

- `Watchlist`: Maintain an `ArrayList<String>` of movie titles.
- `addMovie(String title)`: Add a movie.
- `insertMovie(int index, String title)`: Insert a movie.
- `updateMovie(String oldTitle, String newTitle)`: Update a movie title.
- `removeMovie(String title)`: Remove a movie.
- `searchMovie(String title)`: Search for a movie.
- `displayWatchlist()`: Display all movies.
- `getMovieCount()`: Return the number of movies.
- `Main`: Test all operations.

Concepts covered

ArrayList concept	Relevant problems
`add()`	All eight problems
`add(index, element)`	Library, Shopping Cart, Employee, and others
`get()`	Marks, Employee, and Appointment Management
`set()`	Marks, Employee, To-Do List
`remove()`	All eight problems
`contains()`	All eight problems
`size()`	All eight problems
Iteration using loops	All eight problems
Index validation and exception handling	All eight problems
Numeric calculations using `ArrayList<Integer>`	Student Marks Management

Assessment suggestion: Select any one scenario and students are required to implement the required class and methods, create a `Main` class, and demonstrate all operations with suitable test data.